

**INSTITUTO
FEDERAL**
Santa Catarina

Introdução ao TypeScript

Programação Frontend II

Análise e Desenvolvimento de Sistemas

Prof. Adriano Lima

adriano.lima@ifsc.edu.br



INSTITUTO FEDERAL

Santa Catarina

Câmpus São José

PROGRAMAÇÃO
FRONTEND II

Análise e Desenvolvimento
de Sistemas

cores

#111027

#277756

#16ABCD

#FFF4EC

#C74E23

fontes

Fira Sans Extra Condensed

Ubuntu

Roboto Mono



TypeScript

- superconjunto do JS
- adiciona tipagem estática
 - recursos de tipos melhoram OO
- detecta erros precocemente
- melhora a legibilidade e a manutenção do código
- contribui para a escalabilidade do projeto
- aprimora a contribuição de IDEs



Alternativas ao TypeScript



CoffeeScript

- lançada em 2009
- alternativa mais concisa e expressiva ao JavaScript
- tornar a sintaxe do JS mais parecida com a do Ruby ou do Python



Dart

- lançada em 2011
- alternativa ao JS para aplicativos web, móveis e desktop
- sintaxe semelhante ao Java e ao C#



elm

- lançada em 2012
- projetado para a construção de aplicações web
- modelo de programação funcional
- sintaxe inspirada no Haskell



Preparação

pré-requisito node.js com npm

instalação global

1. instalar typescript via npm
 - `npm install -g typescript`
2. testar se está tudo certo
 - `tsc --version`

instalação local

1. instalar typescript via npm
 - `npm install -D typescript`
2. adicionar o compilador à lista de scripts em `package.json`
 - ```
"scripts": {
 "tsc": "tsc"
}
```
3. testar se está tudo certo
  - `npm run tsc -- --version`



# Preparação

## configuração

### 1. criar arquivo tsconfig.json

- manualmente
- automático
  - `npx tsc --init`

### 2. instalar/atualizar os módulos do projeto

- `npm install`

## exemplo

### tsconfig.json

```
{
 "compilerOptions": {
 "target": "ES2016",
 "outDir": "./dist/js",
 "noEmitOnError": true,
 "module": "commonjs",
 "moduleResolution": "node",
 "strict": true
 },
 "include": [
 "./src/**/*.ts"
]
}
```



# Preparação

## compilação

### 1. por arquivo

- `tsc arquivo.ts` (global)
- `npm run tsc -- arquivo.ts` (local)

### 2. automática

- `tsc -w` (global)
- `npm run tsc -- -w` (local)



# Tipos

## definição

- um conjunto de valores e o que pode ser feito com eles

## exemplos

- booleano
  - **valores** true, false
  - **operações** ||, &&, !, etc
- número
  - **valores** 0, 1, 2, 3, 4, 5, 6, 7, 8, 9
  - **operações** +, -, \*, /, %, .toFixed, .toPrecision, .toString, etc
- string
  - **valores** *todas as strings*
  - **operações** +, .concat, .toUpperCase, .slice, .includes, .length, etc



# Tipos

## definição

- um conjunto de valores e o que pode ser feito com eles
- podem ser definidos explicitamente
  - valor: tipo
- podem ser inferidos

## exemplos

```
// tipos definidos explicitamente
let a: number = 1
let b: string = 'hello'
let c: boolean[] = [true, false]
```

```
// tipos inferidos
let a = 1
let b = 'hello'
let c = [true, false]
```

# Tipos

## primitivos

### ■ any

```
let a: any = 666
let b: any = ['perigo']
let c = a + b // atribuicao implicita
```

### ■ unknown

```
let a: unknown = 30
let b = a === 123
let c = a + 10 // Error TS2571: Object is of type
'unknown'.

if (typeof a === 'number') {
 let d = a + 10 // number
}
```

### ■ boolean

```
let temSaldo: boolean = true;
let verdadeiro: true = true;
```

### ■ number

```
let numero = 10;
let idade: number = 34;
let media: 6.5 = 6.5;
```

### ■ string

```
let nome = 'João';
let sobrenome: string = 'Silva';
```

# Objetos

## declarando

### ■ anotando como object

```
let obj: object = { prop: 12 };
obj.prop; // Error TS2339: Property 'prop' does not
exist on type 'object'.
```

### ■ como object literal

```
let obj = { prop: 12 };
console.log(obj.prop); // 12
obj.outraProp = 123; // Error TS2339: Property
'outraProp' does not exist on type 'object'.
```

### ■ definindo o formato do objeto

```
let obj: {
 prop: number // obrigatória
 nome?: string // opcional
 [chave: number]: boolean // qualquer quantidade;
 [chave: T]: (index signature)
}
obj = { prop: 12, nome: 'João', 1: true, 2: false }
```

### ■ como objeto vazio

```
let obj = {};
obj = { prop: 12 };
obj = { prop: 12, nome: 'João' };
```



# Tipos Personalizados (Type Aliases)

## o que são

- "apelidos" para tipos
- ajudam no entendimento
- não são inferidos; devem ser definidos explicitamente
- podem ser substituídos pelo tipo original
- escopo de bloco

## exemplos

```
type Matricula = number
let novaMatricula: Matricula = 202421232;

type Pessoa = {
 matricula: Matricula;
 nome: string;
 funcao: string;
}

let novoAluno: Pessoa = {
 matricula: novaMatricula,
 nome: 'João',
 funcao: 'Aluno'
}
```



# União e Interseção

## união

- operador |
- permite que uma variável aceite um valor de mais de um tipo

```
let id: string | number;
```

```
id = "ABC123"; // Válido
```

```
id = 456; // Também válido
```

## interseção

- operador &
- combina múltiplos tipos em um único tipo

```
type Pessoa = {
 nome: string;
 idade: number
};
```

```
type Funcionario = {
 id: number;
 departamento: string
};
```

```
type PessoaFuncionario = Pessoa & Funcionario;
```

```
const funcionário: PessoaFuncionario = {
 nome: "Alice",
 idade: 30,
 id: 123,
 departamento: "Engenharia"
};
```



# Arrays

## declarando

### ■ sem anotação (tipo inferido)

```
let numeros = [1, 2, 3];
var letras = ['a', 'b'];
```

### ■ anotando o tipo

```
let numeros: Array<number>;
var letras: Array<string>;

// -----
let numeros: number[];
var letras: string[];
```

### ■ vazio

```
let vazio = []; // any[]
vazio.push(1); // number[]
vazio.push('a'); // (number | string)[]
```

### ■ apenas leitura

```
let numeros: readonly number[] = [1, 2, 3];

numeros.push(4); // Error TS2339: Property 'push'
does not exist on type 'readonly number[]'.
```



# Enumeráveis (Enums)

## o que são

- conjunto de constantes nomeadas
- os valores podem ser do tipo number ou string

## exemplos

```
enum Idioma {
 INGLES,
 ESPANHOL,
 RUSSO
}
```

```
enum Idioma {
 INGLES = 0,
 ESPANHOL = 1,
 RUSSO = 2
}
```

```
enum Idioma {
 INGLES = "inglês",
 ESPANHOL = "espanhol",
 RUSSO = "russo"
}
```



# Enumeráveis (Enums)

## o que são

- conjunto de constantes nomeadas
- os valores podem ser do tipo number ou string

## exemplos

```
enum TipoFuncao {
 ALUNO = 'Aluno',
 PROFESSOR = 'Professor',
 FUNCIONARIO = 'Funcionário'
}

type Pessoa = {
 matricula: Matricula;
 nome: string;
 funcao: TipoFuncao;
}

let novoAluno: Pessoa = {
 matricula: 123,
 nome: 'João',
 funcao: TipoFuncao.ALUNO
}
```



# Funções

## declarando

### ■ funções nomeadas

```
function saudacao(nome: string): string {
 return 'olá ' + nome
}
```

### ■ expressão de função

```
let saudacao = function (nome: string): string {
 return 'olá ' + nome
}
```

### ■ função seta

```
let saudacao = (nome: string): string => {
 return 'olá ' + nome
}
```

```
let saudacao = (nome: string): string =>
 'olá ' + nome
```

# Parâmetros

## opcionais

- marcados com o símbolo ?
- devem vir depois dos obrigatórios

## exemplos

```
function log(message: string, userId?: string) {
 let time = new Date().toLocaleTimeString();
 console.log(time, message, userId || "Não conectado");
}

log("Página carregada");
// Logs "12:38:31 PM Página carregada Não conectado"

log("Usuário conectado em", "da763be");
// Logs "12:38:31 PM Usuário conectado em da763be"
```



# Parâmetros

## opcionais

- marcados com o símbolo ?
- devem vir depois dos obrigatórios
- podem ter valores padrão
  - neste caso, sem ?

## exemplos

```
function log(message: string, userId = "Não conectado") {
 let time = new Date().toLocaleTimeString();
 console.log(time, message, userId);
}
```

```
log("Página carregada");
// Logs "12:38:31 PM Página carregada Não conectado"
```

```
log("Usuário conectado em", "da763be");
// Logs "12:38:31 PM Usuário conectado em da763be"
```



# Parâmetros

## parâmetros rest

- permitem qualquer número de parâmetros (*funções variádicas*)
- polimorfismo de funções

## exemplos

```
function calcularArea(...args: number[]): number {

 if (args.length === 1) {
 return Math.PI * args[0] ** 2;
 } else if (args.length === 2) {
 return args[0] * args[1];
 } else {
 throw new Error("Número inválido de argumentos");
 }
}

console.log(calcularArea(5));
console.log(calcularArea(4, 6));

function soma(...numbers: number[]): number {
 return numbers.reduce((total, n) => total + n, 0);
}

soma(1, 2, 3);
```



# Funções

## definindo o tipo

- usar type
- utilizar apenas código de tipo, não utilizar valores
- criar assinatura do tipo resumida
- criar assinatura de tipo completa

## exemplos

```
type Operacao = (a: number, b: number) => number;

const soma: Operacao = (a, b) => a + b;
const subtracao: Operacao = (a, b) => a - b;
const multiplicacao: Operacao = (a, b) => a * b;
const divisao: Operacao = (a, b) => a / b;

type Operacao = {
 (a: number, b: number): number
};
```



# Funções

## sobrecarga

- permite definir diferentes assinaturas para a mesma função
- composta de
  - uma ou mais assinaturas da função sem corpo
  - uma única implementação compatível com todas as assinaturas

## exemplos

```
type Operacao = {
 (a: number, b: number): number
 (a: string, b: string): string
};

const combinar: Operacao = (a, b): unknown => {
 if (typeof a === "string" && typeof b === "string") {
 return a.concat(b);
 }
 if (typeof a === "number" && typeof b === "number") {
 return a + b;
 }
};
```

# Funções

## sobrecarga

- permite definir diferentes assinaturas para a mesma função
- composta de
  - uma ou mais assinaturas da função sem corpo
  - uma única implementação compatível com todas as assinaturas

## exemplos

```
function formatar(data: Date): string;
function formatar(texto: string, maiuscula: boolean):
string;

function formatar(a: Date | string, b?: boolean): string
{
 if (a instanceof Date) {
 return a.toISOString();
 }
 if (typeof a === "string") {
 return b ? a.toUpperCase() : a.toLowerCase();
 }
 return "";
}

console.log(formatar(new Date())); // ex:
2025-05-05T18:23:00.000Z
console.log(formatar("Olá Mundo", true)); // OLÁ MUNDO
console.log(formatar("Olá Mundo", false)); // olá mundo
```



# Generics

## o que são

- parâmetro de tipo polimórfico
- usado quando o tipo é desconhecido antes do uso
- impõe uma restrição de nível de tipo em vários lugares
- define o tipo "genérico" <T>

## exemplos

```
// JS
function filter(array, f) {

 let result = []

 for (let i = 0; i < array.length; i++) {
 let item = array[i]
 if (f(item)) {
 result.push(item)
 }
 }

 return result
}

// TS
type Filter = <T>(
 array: T[],
 f: (item: T) => boolean
) => T[];
```



# Generics

## o que são

- parâmetro de tipo polimórfico
- usado quando o tipo é desconhecido antes do uso
- impõe uma restrição de nível de tipo em vários lugares
- define o tipo "genérico" <T>

## exemplos

```
type Filter = <T>(
 array: T[],
 f: (item: T) => boolean
) => T[];

let filter: Filter = (array, f) => {
 let result = []

 for (let i = 0; i < array.length; i++) {

 let item = array[i]

 if (f(item)) {
 result.push(item)
 }
 }

 return result
}
```



# Classes

## definição

- definidas como em JS
- suporte para:
  - iniciadores de propriedades
  - modificadores de visibilidade
  - abstração
  - interfaces
  - polimorfismo

## exemplos

```
class Retangulo {

 constructor(largura: number, altura: number) {
 this.largura = largura;
 this.altura = altura;
 }

 area(): number {
 return this.largura * this.altura;
 }

 perimetro(): number {
 return 2*(this.largura + this.altura);
 }
}
```



# Classes

## definição

- definidas como em JS
- suporte para:
  - **iniciadores de propriedades**
  - modificadores de visibilidade
  - abstração
  - interfaces
  - polimorfismo

## exemplos

```
class Retangulo {
 _largura: number;
 _altura: number;

 constructor(largura: number, altura: number) {
 this._largura = largura;
 this._altura = altura;
 }

 get largura(): number {
 return this._largura;
 }

 get altura(): number {
 return this._altura;
 }
}
```



# Classes

## definição

- definidas como em JS
- suporte para:
  - iniciadores de propriedades
  - **modificadores de visibilidade**
  - abstração
  - interfaces
  - polimorfismo

## exemplos

```
class Retangulo {
 private _largura: number;
 private _altura: number;

 constructor(largura: number, altura: number) {
 this._largura = largura;
 this._altura = altura;
 }

 protected area(): number {
 return this._largura * this._altura;
 }

 perimetro(): number {
 return 2*(this._largura + this._altura);
 }
}
```



# Classes

## definição

- definidas como em JS
- suporte para:
  - iniciadores de propriedades
  - **modificadores de visibilidade**
  - abstração
  - interfaces
  - polimorfismo

## exemplos

```
class Retangulo {

 constructor(
 private _largura: number,
 private _altura: number
) {}

 protected area(): number {
 return this._largura * this._altura;
 }

 public perimetro(): number {
 return 2*(this._largura + this._altura);
 }
}
```



# Classes

## definição

- definidas como em JS
- suporte para:
  - iniciadores de propriedades
  - modificadores de visibilidade
  - **abstração**
  - interfaces
  - polimorfismo

## exemplos

```
abstract class Transporte {
 constructor(public nome: string) {}
 abstract mover(): void;
 parar(): void {
 console.log(`${this.nome} parou.`);
 }
}

class Carro extends Transporte {
 mover(): void {
 console.log(`${this.nome} está dirigindo.`);
 }
}

class Aviao extends Transporte {
 mover(): void {
 console.log(`${this.nome} está voando.`);
 }
}
```



# Classes

## definição

- definidas como em JS
- suporte para:
  - iniciadores de propriedades
  - modificadores de visibilidade
  - abstração
  - **interfaces**
  - polimorfismo

## exemplos

```
interface Animal {
 readonly nome: string;
 comer(comida: string): void;
 dormir(horas: number): void;
}

interface Felino {
 miar(): void;
}

class Gato implements Animal, Felino {
 nome: string = 'Bichano';
 comer(comida: string) {
 console.log('Comi ', comida, '. Mmm!');
 };
 dormir(horas: number) {
 console.log('Dormi por', horas, 'horas');
 };
 miar() { console.log('Miau!'); };
}
```



# Classes

## definição

- definidas como em JS
- suporte para:
  - iniciadores de propriedades
  - modificadores de visibilidade
  - abstração
  - interfaces
  - **polimorfismo**

## exemplos

```
interface Queue<T> {
 enqueue(item: T): void;
 dequeue(): T | undefined;
 size(): number;
}

class ArrayQueue<T> implements Queue<T> {
 private items: T[] = [];
 enqueue(item: T): void {
 this.items.push(item);
 }
 dequeue(): T | undefined {
 return this.items.shift();
 }
 size(): number {
 return this.items.length;
 }
}
```



# Classes

## definição

- definidas como em JS
- suporte para:
  - iniciadores de propriedades
  - modificadores de visibilidade
  - abstração
  - interfaces
  - **polimorfismo**
- são estruturalmente tipadas
  - classes com a mesma forma são compatíveis
  - **exceção** classes com campos privados (`private`) ou protegidos (`protected`)

## exemplos

```
class Zebra {
 mover() { console.log("A zebra está trotando!"); }
}

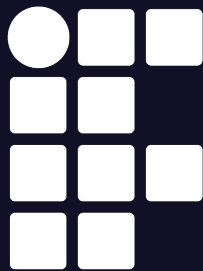
function passear(animal: Zebra) {
 animal.mover();
}

let zebra = new Zebra();
passear(zebra);

class Poodle {
 mover() { console.log("O poodle está andando!"); }
}

let poodle = new Poodle();
passear(poodle);
```





**INSTITUTO  
FEDERAL**  
Santa Catarina

# Introdução ao TypeScript

## Programação Frontend II

Análise e Desenvolvimento de Sistemas

Prof. Adriano Lima

[adriano.lima@ifsc.edu.br](mailto:adriano.lima@ifsc.edu.br)



INSTITUTO FEDERAL

Santa Catarina

Câmpus São José

PROGRAMAÇÃO  
FRONTEND II

Análise e Desenvolvimento  
de Sistemas